

LAPORAN PRAKTIKUM STRUKTUR DATA

Doble Linked List



disusun Oleh:

Tiara Amalia Insani

NIM 2511532019

Dosen Pengampu : DR.Wahyudi, S.T, M.T

Asisten Praktikum : Jovantri Immanuel Gulo

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadirat Allah SWT karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum struktur data mengenai implementasi Linked List dengan baik. Laporan ini dibuat sebagai bentuk pemahaman penulis terhadap materi Double Linked List dalam bahasa pemrograman Java. Dalam praktikum ini penulis membuat program Playlist Musik menggunakan struktur data Double Linked List yang memiliki fitur menambah lagu, menghapus lagu, menampilkan playlist maju dan mundur, serta mencari lagu. Program ini bertujuan agar penulis dapat memahami penggunaan pointer next dan prev pada Double Linked List. Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan ini menjadi lebih baik lagi

Padang, 13 Mei 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I.....	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat.....	1
BAB II	2
PEMBAHASAN	2
2.1 Pengertian Double LinkedList.....	2
2.2 Operasi dalam LinkedList.....	2
2.3 Praktikum	2
2.3.1 Alat dan Bahan	2
2.3.2 Program Class NodeDLL	2
2.3.3 Program Class InsertDLL.....	4
2.3.4 Program Class PenelusuranDLL.....	12
2.3.5 Program Class HapusDLL	15
BAB III.....	25
KESIMPULAN DAN SARAN	25
3.1 Kesimpulan.....	25
3.2 Saran	25
DAFTAR PUSTAKA.....	26
Penjelasan Tugas	27

BAB I

PENDAHULUAN

1.1 Latar Belakang

Linked List merupakan struktur data linear yang terdiri dari kumpulan node yang saling terhubung menggunakan pointer. Pada Double Linked List, setiap node memiliki dua pointer yaitu pointer menuju node berikutnya dan pointer menuju node sebelumnya. Struktur data ini memudahkan proses penelusuran data dari dua arah, yaitu maju dan mundur. Dalam kehidupan sehari-hari, Double Linked List dapat diterapkan pada berbagai sistem seperti playlist musik. Oleh karena itu, praktikum ini dibuat untuk memahami cara kerja Double Linked List melalui implementasi program playlist musik menggunakan bahasa pemrograman Java.

1.2 Tujuan Praktikum

1. Memahami konsep dasar struktur data Double Linked List
2. Memahami penggunaan pointer next dan prev pada node.
3. Mengimplementasikan operasi penambahan data pada Double Linked List.
4. Mengimplementasikan operasi pencarian data pada Double Linked List.
5. Mengimplementasikan operasi penghapusan data pada Double Linked List.

1.3 Manfaat

Praktikum ini bermanfaat untuk meningkatkan pemahaman mengenai struktur data Double Linked List dan cara penerapannya dalam pemrograman Java. Selain itu, praktikum ini membantu mahasiswa memahami proses penambahan, penghapusan, pencarian, dan penelusuran data dari dua arah. Dengan membuat program playlist musik, mahasiswa juga dapat mengetahui penerapan nyata struktur data dalam kehidupan sehari-hari.

BAB II

PEMBAHASAN

2.1 Pengertian Double LinkedList

Double Linked List adalah struktur data linear yang terdiri dari node-node yang saling terhubung. Setiap node memiliki data serta dua pointer, yaitu pointer next untuk menunjuk node berikutnya dan pointer prev untuk menunjuk node sebelumnya. Dengan adanya dua pointer tersebut, proses traversal dapat dilakukan dari depan ke belakang maupun dari belakang ke depan.

2.2 Operasi dalam LinkedList

- a) Traversal → proses menelusuri atau menampilkan seluruh data pada Double Linked List dari node pertama hingga node terakhir.
- b) Insertion → proses menambahkan node baru ke dalam Double Linked List. Penambahan data dapat dilakukan di awal, di akhir, maupun di posisi tertentu, dengan mengatur pointer next dan prev.
- c) Deletion → proses menghapus node pada Double Linked List. Penghapusan dapat dilakukan pada node awal, node akhir, maupun node pada posisi tertentu dengan memperbarui hubungan antar node.
- d) Searching → proses mencari data tertentu pada Double Linked List dengan cara menelusuri node satu per satu hingga data ditemukan.
- e) Updating → proses mengubah data yang terdapat pada node tertentu di dalam Double Linked List tanpa mengubah struktur node lainnya.

2.3 Praktikum

2.3.1 Alat dan Bahan

- 1. Laptop/computer dengan eclipse terinstal.
- 2. Materi praktikum mengenai Double Linked List

2.3.2 Program Class NodeDLL

```
package Pekan6_2511532019;  
  
public class NodeDLL_2511532019 {  
    //mendefinisikan kelas node  
    int data_2019; //data  
    NodeDLL_2511532019 next_2019; //pointer ke next node
```

```

NodeDLL_2511532019 prev_2019; //ponter ke previous node
//konstruktor
public NodeDLL_2511532019 (int data_2019) {
    this.data_2019=data_2019;
    this.next_2019=null;
    this.prev_2019=null;
}
}

```

1. `package pekan6_25115320119;`
kode berada dalam peckage Bernama pekan6_2511532019
2. `public class NodeDLL_2511532019 {`
beris ini mendeklarasikan class Bernama NodeDLL_2511532019 yang dapat diakses dari class lain karena menggunakan keyword public
3. `int data_2019;`
membuat variabel bertipe integer Bernama data_2019 untuk menyimpan nilai pada node.
4. `NodeDLL_2511532019 next_2019;`
Membuat variabel referensi Bernama next_2019 yang menunjuk ke node berikutnya
5. `NodeDLL_2511532019 prev_2019;`
Membuat pointer prev_2019 untuk menghubungkan node ke node sebelumnya pada Double Linked List.
6. `public NodeDLL_2511532019(int data_2019) {`
baris ini mendeklarasikan konstruktor class NodeDLL_2511532019 yang menerima parameter data_2019
7. `this.data_2019=data_2019;`
mengisi variabel data_2019 milik objek dengan nilai parameter data_2019 yang diberikan saat objek dibuat
8. `this.next_2019=null;`
mengatur pointer next_2019 menjadi null, artinya node belum terhubung ke node lain
9. `this.prev_2019=null;`
Menginisialisasi pointer prev_2019 menjadi null, artinya node belum terhubung ke node sebelumnya.

2.3.3 Program Class InsertDLL

```
package Pekan6_2511532019;

public class InsertDLL_2511532019 {
    //menambah node di awal DLL
    static NodeDLL_2511532019 insertBegin_2019(NodeDLL_2511532019
head_2019, int data_2019) {
        //buat node baru
        NodeDLL_2511532019 new_node_2019=new
NodeDLL_2511532019(data_2019);
        //jadikan pointer next nya head
        new_node_2019.next_2019=head_2019;
        //jadikan pointer prev head ke new_node
        if (head_2019 != null) {
            head_2019.prev_2019=new_node_2019;
        }
        return new_node_2019;
    }
    //fungsi menambahkan node di akhir
    public static NodeDLL_2511532019
insertEnd_2019(NodeDLL_2511532019 head_2019, int newData_2019) {
        //buat node baru
        NodeDLL_2511532019 newNode_2019=new NodeDLL_2511532019
(newData_2019);
        //jika dll nul jadikan head
        if (head_2019==null) {
            head_2019=newNode_2019;
        }
        else {
            NodeDLL_2511532019 curr_2019=head_2019;
            while (curr_2019.next_2019 != null) {
                curr_2019=curr_2019.next_2019;
            }
            curr_2019.next_2019=newNode_2019;
            newNode_2019.prev_2019=curr_2019;
        }
        return head_2019;
    }
    //fungsi menambah node di posisi tertentu
    public static NodeDLL_2511532019 insertAtPosition_2019
(NodeDLL_2511532019 head_2019, int pos_2019, int new_data_2019) {
        //buat node baru
        NodeDLL_2511532019 new_node_2019 = new NodeDLL_2511532019
(new_data_2019);
        if (pos_2019==1) {
            new_node_2019.next_2019=head_2019;
            if (head_2019 != null) {
                head_2019.prev_2019=new_node_2019;}
            head_2019=new_node_2019;
            return head_2019;}
        NodeDLL_2511532019 curr_2019 =head_2019;
        for (int i=1;i<pos_2019-1 && curr_2019 != null;++i) {
            curr_2019=curr_2019.next_2019;}
        if (curr_2019==null) {
            System.out.println("Posisi tidak ada");
            return head_2019;}
    }
```

```

        new_node_2019.prev_2019=curr_2019;
        new_node_2019.next_2019=curr_2019.next_2019;
        curr_2019.next_2019=new_node_2019;
        if (new_node_2019.next_2019 != null) {
            new_node_2019.next_2019.prev_2019=new_node_2019;}
        return head_2019;
    }
    public static void printList_2019 (NodeDLL_2511532019 head_2019)
    {
        NodeDLL_2511532019 curr_2019=head_2019;
        while (curr_2019!=null) {
            System.out.print(curr_2019.data_2019 + " <-> ");
            curr_2019=curr_2019.next_2019;
        }
        System.out.println();
    }
    public static void main (String[] args) {
        //membuat dll 2 <-> 3 <-> 5
        NodeDLL_2511532019 head_2019 = new NodeDLL_2511532019(2);
        head_2019.next_2019=new NodeDLL_2511532019(3);
        head_2019.next_2019.prev_2019=head_2019;
        head_2019.next_2019.next_2019=new NodeDLL_2511532019(5);

        head_2019.next_2019.next_2019.prev_2019=head_2019.next_2019;
        //cetak DLL awal
        System.out.print("DLL Awal: ");
        printList_2019(head_2019);
        //tambah 1 di awal
        head_2019= insertBegin_2019 (head_2019, 1);
        System.out.print("Simpul 1 ditambah di awal: ");
        printList_2019(head_2019);
        //tambah 6 di akhir
        System.out.print("Simpul 6 ditambah di akhir: ");
        int data_2019=6;
        head_2019=insertEnd_2019(head_2019, data_2019);
        printList_2019(head_2019);
        //menambah node 4 di posisi 4
        System.out.print("Tambah node 4 di posisi 4: ");
        int data2_2019=4;
        int pos_2019=4;
        head_2019=insertAtPosition_2019(head_2019, pos_2019,
data2_2019);
        printList_2019(head_2019);
    }
}

```

```

public class InsertDLL_2511532019 {
    //menambah node di awal DLL
    static NodeDLL_2511532019 insertBegin_2019(NodeDLL_2511532019
head_2019, int data_2019) {
        //buat node baru
        NodeDLL_2511532019 new_node_2019=new
NodeDLL_2511532019(data_2019);
        //jadikan pointer next nya head
        new_node_2019.next_2019=head_2019;
    }
}

```



```

        //jadikan pointer prev head ke new_node
        if (head_2019 != null) {
            head_2019.prev_2019=new_node_2019;
        }
        return new_node_2019;
    }
}

```

- public class InsertDLL_2511532019 {
→Membuat class bernama InsertDLL_2511532019.
- static NodeDLL_2511532019 insertBegin_2019(...) {
→Method untuk menambahkan node di awal DLL.
- NodeDLL_2511532019 new_node_2019=new
NodeDLL_2511532019(data_2019);
→Membuat node baru dengan data yang diberikan.
- new_node_2019.next_2019=head_2019;
→Pointer next node baru diarahkan ke head.
- if (head_2019 != null) {
→Mengecek apakah DLL tidak kosong.
- head_2019.prev_2019=new_node_2019;
→Pointer prev head diarahkan ke node baru.
- return new_node_2019;
→Mengembalikan node baru sebagai head.

```

//fungsi menambahkan node di akhir
public static NodeDLL_2511532019
insertEnd_2019(NodeDLL_2511532019 head_2019, int newData_2019) {
    //buat node baru
    NodeDLL_2511532019 newNode_2019=new NodeDLL_2511532019
(newData_2019);
    //jika dll nul jadikan head
    if (head_2019==null) {
        head_2019=newNode_2019;
    }
    else {
        NodeDLL_2511532019 curr_2019=head_2019;
        while (curr_2019.next_2019 != null) {
            curr_2019=curr_2019.next_2019;
        }
        curr_2019.next_2019=newNode_2019;
        newNode_2019.prev_2019=curr_2019;
    }
    return head_2019;
}
}

```

- `public static NodeDLL_2511532019 insertEnd_2019(...) {`
→ Method menambah node di akhir DLL.
- `NodeDLL_2511532019 newNode_2019=new`
`NodeDLL_2511532019(newData_2019);`
→ Membuat node baru.
- `if (head_2019==null) {`
→ Mengecek apakah DLL kosong.
- `head_2019=newNode_2019;`
→ Node baru dijadikan head.
- `else {`
→ Kondisi jika DLL tidak kosong.
- `NodeDLL_2511532019 curr_2019=head_2019;`
→ Variabel traversal mulai dari head.
- `while (curr_2019.next_2019 != null) {`
→ Menelusuri node sampai node terakhir.
- `curr_2019=curr_2019.next_2019;}`
→ Pointer berpindah ke node berikutnya.
- `curr_2019.next_2019=newNode_2019;`
→ Node terakhir dihubungkan ke node baru.
- `newNode_2019.prev_2019=curr_2019;}`
→ Pointer prev node baru diarahkan ke node sebelumnya.
- `return head_2019;`
→ Mengembalikan head DLL.

```
//fungsi menambah node di posisi tertentu
public static NodeDLL_2511532019 insertAtPosition_2019
(NodeDLL_2511532019 head_2019, int pos_2019, int new_data_2019) {
    //buat node baru
    NodeDLL_2511532019 new_node_2019 = new NodeDLL_2511532019
(new_data_2019);
    if (pos_2019==1) {
        new_node_2019.next_2019=head_2019;
        if (head_2019 != null) {
            head_2019.prev_2019=new_node_2019;
        }
        head_2019=new_node_2019;
        return head_2019;}
}
```

- `public static NodeDLL_2511532019 insertAtPosition_2019(...) {`
→Method menambah node berdasarkan posisi.
- `NodeDLL_2511532019 new_node_2019 = new`
`NodeDLL_2511532019(new_data_2019);`
→Membuat node baru.
- `if (pos_2019==1) {`
→Mengecek apakah posisi adalah posisi pertama.
- `new_node_2019.next_2019=head_2019;`
→Pointer next node baru diarahkan ke head lama.
- `if (head_2019 != null) {`
→Mengecek apakah DLL tidak kosong.
- `head_2019.prev_2019=new_node_2019;}`
→Pointer prev head diarahkan ke node baru.
- `head_2019=new_node_2019;`
→Node baru dijadikan head.
- `return head_2019;}`
→Mengembalikan head DLL.

```
NodeDLL_2511532019 curr_2019 =head_2019;
for (int i=1;i<pos_2019-1 && curr_2019 != null;++i) {
    curr_2019=curr_2019.next_2019;}
if (curr_2019==null) {
    System.out.println("Posisi tidak ada");
    return head_2019;}
new_node_2019.prev_2019=curr_2019;
new_node_2019.next_2019=curr_2019.next_2019;
curr_2019.next_2019=new_node_2019;
if (new_node_2019.next_2019 != null) {
    new_node_2019.next_2019.prev_2019=new_node_2019;}
return head_2019;
}
```

- `NodeDLL_2511532019 curr_2019 =head_2019;`
→Variabel traversal mulai dari head.
- `for (int i=1;i<pos_2019-1 && curr_2019 != null;++i) {`
→Perulangan menuju posisi sebelum node yang akan disisipkan.
- `curr_2019=curr_2019.next_2019;}`
→Pointer berpindah ke node berikutnya.

- `if (curr_2019==null) {`
→Mengecek apakah posisi tidak ditemukan.
- `System.out.println("Posisi tidak ada");`
→Menampilkan pesan error posisi.
- `return head_2019;}`
→Mengembalikan head DLL.
- `new_node_2019.prev_2019=curr_2019;`
→Pointer prev node baru diarahkan ke node sebelumnya.
- `new_node_2019.next_2019=curr_2019.next_2019;`
→Pointer next node baru diarahkan ke node sesudahnya.
- `curr_2019.next_2019=new_node_2019;`
→Node sebelumnya dihubungkan ke node baru.
- `if (new_node_2019.next_2019 != null) {`
→Mengecek apakah ada node setelah node baru.
- `new_node_2019.next_2019.prev_2019=new_node_2019;}`
→Pointer prev node setelahnya diarahkan ke node baru.
- `return head_2019;}`
→Mengembalikan head DLL.

```
public static void printList_2019 (NodeDLL_2511532019 head_2019)
{
    NodeDLL_2511532019 curr_2019=head_2019;
    while (curr_2019!=null) {
        System.out.print(curr_2019.data_2019 + " <-> ");
        curr_2019=curr_2019.next_2019;
    }
    System.out.println();
}
```

- `public static void printList_2019 (...)` {
→Method untuk mencetak DLL.
- `NodeDLL_2511532019 curr_2019=head_2019;`
→Variabel traversal mulai dari head.
- `while (curr_2019!=null) {`
→Perulangan traversal sampai node terakhir.
- `System.out.print(curr_2019.data_2019 + " <-> ");`
→Menampilkan data node.

- `curr_2019=curr_2019.next_2019;`
→ Pointer berpindah ke node berikutnya.
- `System.out.println();`
→ Membuat baris baru.

```
public static void main (String[] args) {
    //membuat dll 2 <-> 3 <-> 5
    NodeDLL_2511532019 head_2019 = new NodeDLL_2511532019(2);
    head_2019.next_2019=new NodeDLL_2511532019(3);
    head_2019.next_2019.prev_2019=head_2019;
    head_2019.next_2019.next_2019=new NodeDLL_2511532019(5);

    head_2019.next_2019.next_2019.prev_2019=head_2019.next_2019;
```

- `public static void main (String[] args) {`
→ Method utama program.
- `NodeDLL_2511532019 head_2019 = new NodeDLL_2511532019(2);`
→ Membuat node pertama bernilai 2.
- `head_2019.next_2019=new NodeDLL_2511532019(3);`
→ Menambahkan node bernilai 3.
- `head_2019.next_2019.prev_2019=head_2019;`
→ Pointer prev node 3 diarahkan ke node 2.
- `head_2019.next_2019.next_2019=new NodeDLL_2511532019(5);`
→ Menambahkan node bernilai 5.
- `head_2019.next_2019.next_2019.prev_2019=head_2019.next_2019;`
→ Pointer prev node 5 diarahkan ke node 3.

```
//cetak DLL awal
System.out.print("DLL Awal: ");
printList_2019(head_2019);
//tambah 1 di awal
head_2019= insertBegin_2019 (head_2019, 1);
System.out.print("Simpul 1 ditambah di awal: ");
printList_2019(head_2019);
//tambah 6 di akhir
System.out.print("Simpul 6 ditambah di akhir: ");
int data_2019=6;
head_2019=insertEnd_2019(head_2019, data_2019);
printList_2019(head_2019);
//menambah node 4 di posisi 4
System.out.print("Tambah node 4 di posisi 4: ");
int data2_2019=4;
int pos_2019=4;
head_2019=insertAtPosition_2019(head_2019, pos_2019,
data2_2019);
printList_2019(head_2019);
```

```
}  
}
```

- `System.out.print("DLL Awal: ");`
→ Menampilkan teks DLL awal.
- `printList_2019(head_2019);`
→ Mencetak isi DLL.
- `head_2019= insertBegin_2019(head_2019, 1);`
→ Menambahkan node bernilai 1 di awal DLL.
- `System.out.print("Simpul 1 ditambah di awal: ");`
→ Menampilkan teks hasil penambahan.
- `printList_2019(head_2019);`
→ Mencetak DLL terbaru.
- `System.out.print("Simpul 6 ditambah di akhir: ");`
→ Menampilkan teks penambahan.
- `int data_2019=6;`
→ Variabel data bernilai 6.
- `head_2019=insertEnd_2019(head_2019, data_2019);`
→ Menambahkan node 6 di akhir DLL.
- `printList_2019(head_2019);`
→ Mencetak DLL terbaru.
- `System.out.print("Tambah node 4 di posisi 4: ");`
→ Menampilkan teks penambahan.
- `int data2_2019=4;`
→ Variabel data bernilai 4.
- `int pos_2019=4;`
→ Variabel posisi bernilai 4.
- `head_2019=insertAtPosition_2019(head_2019, pos_2019, data2_2019);`
→ Menambahkan node 4 pada posisi ke-4.
- `printList_2019(head_2019);}}`
→ Mencetak DLL terbaru.

Output

```

DLL Awal: 2 <-> 3 <-> 5 <->
Simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

2.3.4 Program Class PenelusuranDLL

```

package Pekan6_2511532019;

public class PenelusuranDLL_2511532019 {
    //fungsi penelusuran maju
    static void forwardTraversal_2019(NodeDLL_2511532019 head_2019) {
        //memulai penelusuran dari head
        NodeDLL_2511532019 curr_2019=head_2019;
        //lanjutkan sampai akhir
        while (curr_2019 != null) {
            //print data
            System.out.print(curr_2019.data_2019 + " <-> ");
            //pindah ke node berikutnya
            curr_2019=curr_2019.next_2019;
        }
        //print spasi
        System.out.println();
    }
    //fungsi penelusuran mundur
    static void backwardTraversal_2019(NodeDLL_2511532019 tail_2019)
    {
        //mulai dari akhir
        NodeDLL_2511532019 curr_2019=tail_2019;
        //lanjut sampai head
        while (curr_2019 != null) {
            //cetak data
            System.out.print(curr_2019.data_2019 + " <-> ");
            //pindah ke node sebelumnya
            curr_2019=curr_2019.prev_2019;
        }
        //cetak spasi
        System.out.println();
    }
    public static void main (String[] args) {
        //cetak DLL
        NodeDLL_2511532019 head_2019 = new NodeDLL_2511532019(1);
        NodeDLL_2511532019 second_2019=new NodeDLL_2511532019(2);
        NodeDLL_2511532019 third_2019=new NodeDLL_2511532019(3);

        head_2019.next_2019=second_2019;
        second_2019.prev_2019=head_2019;
        second_2019.next_2019=third_2019;
        third_2019.prev_2019=second_2019;

        System.out.println("Penelusuran maju: ");
        forwardTraversal_2019(head_2019);

        System.out.println("Penelusuran mundur: ");
        backwardTraversal_2019 (third_2019);
    }
}

```

```
}
}
```

```
public class PenelusuranDLL_2511532019 {
    //fungsi penelusuran maju
    static void forwardTraversal_2019(NodeDLL_2511532019 head_2019) {
        //memulai penelusuran dari head
        NodeDLL_2511532019 curr_2019=head_2019;
        //lanjutkan sampai akhir
        while (curr_2019 != null) {
            //print data
            System.out.print(curr_2019.data_2019 + " <-> ");
            //pindah ke node berikutnya
            curr_2019=curr_2019.next_2019;
        }
        //print spasi
        System.out.println();
    }
}
```

- public class PenelusuranDLL_2511532019 {
→Membuat class bernama PenelusuranDLL_2511532019.
- static void forwardTraversal_2019(...) {
→Method untuk menelusuri DLL dari depan ke belakang.
- NodeDLL_2511532019 curr_2019=head_2019;
→Variabel curr_2019 mulai dari head.
- while (curr_2019 != null) {
→Perulangan selama node tidak null.
- System.out.print(curr_2019.data_2019 + " <-> ");
→Menampilkan data node.
- curr_2019=curr_2019.next_2019;}
→Pointer berpindah ke node selanjutnya.
- System.out.println();
→Mencetak pindah baris.

```
//fungsi penelusuran mundur
static void backwardTraversal_2019(NodeDLL_2511532019 tail_2019)
{
    //mulai dari akhir
    NodeDLL_2511532019 curr_2019=tail_2019;
    //lanjut sampai head
    while (curr_2019 != null) {
        //cetak data
        System.out.print(curr_2019.data_2019 + " <-> ");
        //pindah ke node sebelumnya
        curr_2019=curr_2019.prev_2019;
    }
}
```



```

    }
    //cetak spasi
    System.out.println();
}

```

- static void backwardTraversal_2019(...) {
→Method untuk traversal dari belakang ke depan.
- NodeDLL_2511532019 curr_2019=tail_2019;
→Variabel traversal dimulai dari tail.
- while (curr_2019 != null) {
→Perulangan selama node tidak null.
- System.out.print(curr_2019.data_2019 + " <-> ");
→Menampilkan data node.
- curr_2019=curr_2019.prev_2019;}
→Pointer berpindah ke node sebelumnya.
- System.out.println();}
→Mencetak pindah baris.

```

public static void main (String[] args) {
    //cetak DLL
    NodeDLL_2511532019 head_2019 = new NodeDLL_2511532019(1);
    NodeDLL_2511532019 second_2019=new NodeDLL_2511532019(2);
    NodeDLL_2511532019 third_2019=new NodeDLL_2511532019(3);

    head_2019.next_2019=second_2019;
    second_2019.prev_2019=head_2019;
    second_2019.next_2019=third_2019;
    third_2019.prev_2019=second_2019;

    System.out.println("Penelusuran maju: ");
    forwardTraversal_2019(head_2019);

    System.out.println("Penelusuran mundur: ");
    backwardTraversal_2019 (third_2019);
}
}

```

- public static void main (String[] args) {
→Method utama program.
- NodeDLL_2511532019 head_2019 = new NodeDLL_2511532019(1);
→Membuat node pertama bernilai 1.
- NodeDLL_2511532019 second_2019=new NodeDLL_2511532019(2);
→Membuat node kedua bernilai 2.

- NodeDLL_2511532019 third_2019=new NodeDLL_2511532019(3);
→Membuat node ketiga bernilai 3.
- head_2019.next_2019=second_2019;
→Node pertama dihubungkan ke node kedua.
- second_2019.prev_2019=head_2019;
→Pointer prev node kedua diarahkan ke node pertama.
- second_2019.next_2019=third_2019;
→Node kedua dihubungkan ke node ketiga.
- third_2019.prev_2019=second_2019;
→Pointer prev node ketiga diarahkan ke node kedua.
- System.out.println("Penelusuran maju: ");
→Menampilkan teks traversal maju.
- forwardTraversal_2019(head_2019);
→Memanggil method traversal maju.
- System.out.println("Penelususan mundur: ");
→Menampilkan teks traversal mundur.
- backwardTraversal_2019(third_2019);
→Memanggil method traversal mundur mulai dari tail.

Output

```
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelususan mundur:
3 <-> 2 <-> 1 <->
```

2.3.5 Program Class HapusDLL

```
package Pekan6_2511532019;

public class HapusDLL_2511532019 {
    //fungsi menghapus node awal
    public static NodeDLL_2511532019 delHead_2019(NodeDLL_2511532019
head_2019) {
        if (head_2019 == null) {
            return null;
        }
        NodeDLL_2511532019 temp_2019=head_2019;
        head_2019=head_2019.next_2019;
        if (head_2019 != null) {
            head_2019.prev_2019=null;
        }
    }
}
```

```

    }
    return head_2019;
}
//fungsi menghapus di akhir
public static NodeDLL_2511532019 delLast_2019(NodeDLL_2511532019
head_2019) {
    if (head_2019==null) {
        return null;
    }
    if (head_2019.next_2019==null) {
        return null;
    }
    NodeDLL_2511532019 curr_2019=head_2019;
    while (curr_2019.next_2019 != null) {
        curr_2019= curr_2019.next_2019;
    }
    //update pointer previous node
    if (curr_2019.prev_2019 != null) {
        curr_2019.prev_2019.next_2019=null;
    }
    return head_2019;
}
//fungsi menghapus node posisi tertentu
public static NodeDLL_2511532019 delPos_2019(NodeDLL_2511532019
head_2019, int pos_2019) {
    //jika DLL kosong
    if (head_2019==null) {
        return head_2019;
    }
    NodeDLL_2511532019 curr_2019 = head_2019;
    //telusurisampai ke node yang akan dihapus
    for(int i= 1; curr_2019 != null && i<pos_2019; ++i) {
        curr_2019=curr_2019.next_2019;
    }
    //jika posisi tidak ditemukan
    if (curr_2019==null) {
        return head_2019;
    }
    //update pointer
    if (curr_2019.prev_2019 != null) {
        curr_2019.prev_2019.next_2019=curr_2019.next_2019;
    }
    if (curr_2019.next_2019 != null) {
        curr_2019.next_2019.prev_2019=curr_2019.prev_2019;
    }
    //jika yang dihapus head
    if (head_2019== curr_2019) {
        head_2019=curr_2019.next_2019;
    }
    return head_2019;
}
//fungsi mencetak DLL
public static void printList_2019(NodeDLL_2511532019 head_2019) {
    NodeDLL_2511532019 curr_2019=head_2019;
    while (curr_2019 != null) {
        System.out.print(curr_2019.data_2019 + " <-> ");
        curr_2019= curr_2019.next_2019;
    }
}

```

```

    }
    System.out.println();
}
public static void main (String[] args) {
    //buat sebuah DLL
    NodeDLL_2511532019 head_2019=new NodeDLL_2511532019(1);
    head_2019.next_2019=new NodeDLL_2511532019(2);
    head_2019.next_2019.prev_2019=head_2019;
    head_2019.next_2019.next_2019=new NodeDLL_2511532019(3);

    head_2019.next_2019.next_2019.prev_2019=head_2019.next_2019;
    head_2019.next_2019.next_2019.next_2019=new
NodeDLL_2511532019(4);

    head_2019.next_2019.next_2019.next_2019.prev_2019=head_2019.next_
2019.next_2019;
    head_2019.next_2019.next_2019.next_2019.next_2019=new
NodeDLL_2511532019(5);

    head_2019.next_2019.next_2019.next_2019.next_2019.prev_2019=head_
2019.next_2019.next_2019.next_2019;

    System.out.print("DLL Awal: ");
    printList_2019(head_2019);

    System.out.print("Setelah head dihapus: ");
    head_2019=delHead_2019(head_2019);
    printList_2019(head_2019);

    System.out.print("Setelah node terakhir dihapus: ");
    head_2019 = delLast_2019 (head_2019);
    printList_2019(head_2019);

    System.out.print("Menghapus node ke 2: ");
    head_2019=delPos_2019(head_2019, 2);
    printList_2019(head_2019);
}
}

```

```

public class HapusDLL_2511532019 {
    //fungsi menghapus node awal
    public static NodeDLL_2511532019 delHead_2019(NodeDLL_2511532019
head_2019) {
        if (head_2019 == null) {
            return null;
        }
        NodeDLL_2511532019 temp_2019=head_2019;
        head_2019=head_2019.next_2019;
        if (head_2019 != null) {
            head_2019.prev_2019=null;
        }
        return head_2019;
    }
}

```

- `public class HapusDLL_2511532019 {`
→ Membuat class bernama HapusDLL_2511532019.
- `public static NodeDLL_2511532019 delHead_2019(...) {`
→ Method untuk menghapus head DLL.
- `if (head_2019 == null) {`
→ Mengecek apakah DLL kosong.
- `return null;}`
→ Mengembalikan null jika DLL kosong.
- `NodeDLL_2511532019 temp_2019=head_2019;`
→ Menyimpan head lama ke variabel sementara.
- `head_2019=head_2019.next_2019;`
→ Head dipindahkan ke node berikutnya.
- `if (head_2019 != null) {`
→ Mengecek apakah head baru tidak kosong.
- `head_2019.prev_2019=null;}`
→ maka, Pointer prev head baru dibuat null.
- `return head_2019;}`
→ Mengembalikan head baru.

```
//fungsi menghapus di akhir
public static NodeDLL_2511532019 delLast_2019(NodeDLL_2511532019
head_2019) {
    if (head_2019==null) {
        return null;
    }
    if (head_2019.next_2019==null) {
        return null;
    }
    NodeDLL_2511532019 curr_2019=head_2019;
    while (curr_2019.next_2019 != null) {
        curr_2019= curr_2019.next_2019;
    }
    //update pointer previous node
    if (curr_2019.prev_2019 != null) {
        curr_2019.prev_2019.next_2019=null;
    }
    return head_2019;
}
```

- `public static NodeDLL_2511532019 delLast_2019(...) {`
→ Method menghapus node terakhir.

- `if (head_2019==null) {`
→ Mengecek apakah DLL kosong.
- `return null;}`
→ Mengembalikan null jika DLL kosong.
- `if (head_2019.next_2019==null) {`
→ Mengecek apakah DLL hanya memiliki satu node.
- `return null;}`
→ Menghapus node tunggal dengan mengembalikan null.
- `NodeDLL_2511532019 curr_2019=head_2019;`
→ Variabel traversal dimulai dari head.
- `while (curr_2019.next_2019 != null) {`
→ Menelusuri DLL sampai node terakhir.
- `curr_2019= curr_2019.next_2019;}`
→ Pointer berpindah ke node berikutnya.
- `if (curr_2019.prev_2019 != null) {`
→ Mengecek apakah node sebelumnya ada.
- `curr_2019.prev_2019.next_2019=null;}`
→ Pointer next node sebelumnya dibuat null.
- `return head_2019;}`
→ Mengembalikan head DLL.

```
//fungsi menghapus node posisi tertentu
public static NodeDLL_2511532019 delPos_2019(NodeDLL_2511532019
head_2019, int pos_2019) {
    //jika DLL kosong
    if (head_2019==null) {
        return head_2019;
    }
}
```

- `public static NodeDLL_2511532019 delPos_2019(...) {`
→ Method menghapus node di posisi tertentu.
- `if (head_2019==null) {`
→ Mengecek apakah DLL kosong.
- `return head_2019;}`
→ Mengembalikan head jika DLL kosong.

```
NodeDLL_2511532019 curr_2019 = head_2019;
```

```

//telusurisampai ke node yang akan dihapus
for(int i= 1; curr_2019 != null && i<pos_2019; ++i) {
    curr_2019=curr_2019.next_2019;
}
//jika posisi tidak ditemukan
if (curr_2019==null) {
    return head_2019;
}
//update pointer
if (curr_2019.prev_2019 != null) {
    curr_2019.prev_2019.next_2019=curr_2019.next_2019;
}
if (curr_2019.next_2019 != null) {
    curr_2019.next_2019.prev_2019=curr_2019.prev_2019;
}
//jika yang dihapus head
if (head_2019== curr_2019) {
    head_2019=curr_2019.next_2019;
}
return head_2019;
}

```

- NodeDLL_2511532019 curr_2019 = head_2019;
→Variabel traversal dimulai dari head.
- for(int i=1; curr_2019 != null && i<pos_2019; ++i) {
→Perulangan menuju node yang akan dihapus.
- curr_2019=curr_2019.next_2019;}
→Pointer berpindah ke node berikutnya.
- if (curr_2019==null) {
→Mengecek apakah node tujuan tidak ada.
- return head_2019;}
→Mengembalikan head tanpa perubahan.
- if (curr_2019.prev_2019 != null) {
→Mengecek apakah node sebelumnya ada.
- curr_2019.prev_2019.next_2019=curr_2019.next_2019;}
→Node sebelumnya diarahkan ke node sesudah node yang dihapus.
- if (curr_2019.next_2019 != null) {
→Mengecek apakah node sesudahnya ada.
- curr_2019.next_2019.prev_2019=curr_2019.prev_2019;}
→Pointer prev node sesudahnya diarahkan ke node sebelumnya.

- `if (head_2019 == curr_2019) {`
→ Mengecek apakah node yang dihapus adalah head.
- `head_2019 = curr_2019.next_2019;`
→ Head dipindahkan ke node berikutnya.
- `return head_2019;`
→ Mengembalikan head DLL.

```
//fungsi mencetak DLL
public static void printList_2019(NodeDLL_2511532019 head_2019) {
    NodeDLL_2511532019 curr_2019 = head_2019;
    while (curr_2019 != null) {
        System.out.print(curr_2019.data_2019 + " <-> ");
        curr_2019 = curr_2019.next_2019;
    }
    System.out.println();
}
```

- `public static void printList_2019(...) {`
→ Method mencetak isi DLL.
- `NodeDLL_2511532019 curr_2019 = head_2019;`
→ Variabel traversal dimulai dari head.
- `while (curr_2019 != null) {`
→ Perulangan traversal DLL.
- `System.out.print(curr_2019.data_2019 + " <-> ");`
→ Menampilkan data node.
- `curr_2019 = curr_2019.next_2019;`
→ Pointer berpindah ke node berikutnya.
- `System.out.println();`
→ Membuat baris baru.

```
public static void main (String[] args) {
    //buat sebuah DLL
    NodeDLL_2511532019 head_2019 = new NodeDLL_2511532019(1);
    head_2019.next_2019 = new NodeDLL_2511532019(2);
    head_2019.next_2019.prev_2019 = head_2019;
    head_2019.next_2019.next_2019 = new NodeDLL_2511532019(3);

    head_2019.next_2019.next_2019.prev_2019 = head_2019.next_2019;
    head_2019.next_2019.next_2019.next_2019 = new
NodeDLL_2511532019(4);

    head_2019.next_2019.next_2019.next_2019.prev_2019 = head_2019.next_
2019.next_2019;
}
```



```

        head_2019.next_2019.next_2019.next_2019.next_2019=new
NodeDLL_2511532019(5);

        head_2019.next_2019.next_2019.next_2019.next_2019.prev_2019=head_
2019.next_2019.next_2019.next_2019;

```

- public static void main (String[] args) {
→Method utama program.
- NodeDLL_2511532019 head_2019=new NodeDLL_2511532019(1);
→Membuat node pertama bernilai 1.
- head_2019.next_2019=new NodeDLL_2511532019(2);
→Menambahkan node bernilai 2.
- head_2019.next_2019.prev_2019=head_2019;
→Pointer prev node 2 diarahkan ke node 1.
- head_2019.next_2019.next_2019=new NodeDLL_2511532019(3);
→Menambahkan node bernilai 3.
- head_2019.next_2019.next_2019.prev_2019=head_2019.next_2019;
→Pointer prev node 3 diarahkan ke node 2.
- head_2019.next_2019.next_2019.next_2019=new
NodeDLL_2511532019(4);
→Menambahkan node bernilai 4.
- head_2019.next_2019.next_2019.next_2019.prev_2019=head_2019.next_2019.next_2019;
→Pointer prev node 4 diarahkan ke node 3.
- head_2019.next_2019.next_2019.next_2019.next_2019=new
NodeDLL_2511532019(5);
→Menambahkan node bernilai 5.
- head_2019.next_2019.next_2019.next_2019.next_2019.prev_2019=head_2019.next_2019.next_2019.next_2019.next_2019;
→Pointer prev node 5 diarahkan ke node 4.

```

System.out.print("DLL Awal: ");
printList_2019(head_2019);

System.out.print("Setelah head dihapus: ");
head_2019=delHead_2019(head_2019);
printList_2019(head_2019);

```

```

        System.out.print("Setelah node terakhir dihapus: ");
        head_2019 = delLast_2019 (head_2019);
        printList_2019(head_2019);

        System.out.print("Menghapus node ke 2: ");
        head_2019=delPos_2019(head_2019, 2);
        printList_2019(head_2019);
    }
}

```

- System.out.print("DLL Awal: ");
→Menampilkan teks DLL awal.
- printList_2019(head_2019);
→Mencetak isi DLL.
- System.out.print("Setelah head dihapus: ");
→Menampilkan teks setelah penghapusan head.
- head_2019=delHead_2019(head_2019);
→Menghapus node pertama.
- printList_2019(head_2019);
→Mencetak DLL terbaru.
- System.out.print("Setelah node terakhir dihapus: ");
→Menampilkan teks penghapusan node terakhir.
- head_2019 = delLast_2019(head_2019);
→Menghapus node terakhir.
- printList_2019(head_2019);
→Mencetak DLL terbaru.
- System.out.print("Menghapus node ke 2: ");
→Menampilkan teks penghapusan node posisi kedua.
- head_2019=delPos_2019(head_2019, 2);
→Menghapus node pada posisi ke-2.
- printList_2019(head_2019);}}
- Mencetak DLL terbaru.

Output

```
DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->  
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->  
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->  
Menghapus node ke 2: 2 <-> 4 <->
```

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Double Linked List merupakan struktur data yang memungkinkan traversal data dari dua arah menggunakan pointer next dan prev. Implementasi Double Linked List pada program playlist musik dapat digunakan untuk menambah, menghapus, mencari, dan menampilkan data lagu secara maju maupun mundur. Dengan praktikum ini, pemahaman mengenai konsep Double Linked List dan implementasinya dalam Java menjadi lebih baik.

3.2 Saran

1. lebih banyak berlatih dalam mengimplementasikan Double LinkedList agar semakin memahami konsepnya
2. Perlu meningkatkan ketelitian dalam penulisan kode untuk menghindari kesalahan (*error*) saat program dijalankan.

DAFTAR PUSTAKA

- [1] M. T. Goodrich, R. Tamassia, dan M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ: Wiley, 2014.

Penjelasan Tugas

Program Class Lagu

```
package Pekan6_2511532019;

public class Lagu_2511532019 {
    //atribut lagu
    String judul_2019;
    String penyanyi_2019;

    //pointer next dan prev
    Lagu_2511532019 next_2019;
    Lagu_2511532019 prev_2019;

    //konstruktor
    public Lagu_2511532019(String judul_2019, String penyanyi_2019) {
        this.judul_2019=judul_2019;
        this.penyanyi_2019=penyanyi_2019;
        this.next_2019=null;
        this.prev_2019=null;
    }
}
```

```
//atribut lagu
String judul_2019;
String penyanyi_2019;
```

- public class Lagu_2511532019 {
→Membuat class bernama Lagu_2511532019.
- String judul_2019;
→Variabel untuk menyimpan judul lagu.
- String penyanyi_2019;
→Variabel untuk menyimpan nama penyanyi.

```
//pointer next dan prev
Lagu_2511532019 next_2019;
Lagu_2511532019 prev_2019;
```

- Lagu_2511532019 next_2019;
→Pointer menuju node berikutnya.
- Lagu_2511532019 prev_2019;
→Pointer menuju node sebelumnya.

```
//konstruktor
public Lagu_2511532019(String judul_2019, String penyanyi_2019) {
    this.judul_2019=judul_2019;
```

```

        this.penyanyi_2019=penyanyi_2019;
        this.next_2019=null;
        this.prev_2019=null;
    }
}

```

- public Lagu_2511532019(...) {
→Konstruktor untuk menginisialisasi objek lagu.
- this.judul_2019=judul_2019;
→Mengisi atribut judul dengan nilai parameter.
- this.penyanyi_2019=penyanyi_2019;
→Mengisi atribut penyanyi dengan nilai parameter.
- this.next_2019=null;
→Pointer next diinisialisasi null.
- this.prev_2019=null;}}
- Pointer prev diinisialisasi null.

Program Class PlaylistMusik

```

package Pekan6_2511532019;

import java.util.Scanner;

public class PlaylistMusik_2511532019 {

    static Lagu_2511532019 head_2019=null;
    static Lagu_2511532019 tail_2019=null;

    //fungsi tambah lagu
    public static void tambahLagu_2019(String judul_2019, String
    penyanyi_2019) {

        Lagu_2511532019 newLagu_2019=
        new Lagu_2511532019(judul_2019,
    penyanyi_2019);

        //jika DLL kosong
        if(head_2019==null) {
            head_2019=newLagu_2019;
            tail_2019=newLagu_2019;
        }
        else {
            tail_2019.next_2019=newLagu_2019;
            newLagu_2019.prev_2019=tail_2019;
            tail_2019=newLagu_2019;
        }

        System.out.println("Lagu berhasil ditambahkan!");
    }
}

```

```

    }

    //fungsi hapus lagu pertama
    public static void hapusLaguPertama_2019() {

        //jika kosong
        if(head_2019==null) {
            System.out.println("Playlist kosong");
            return;
        }

        System.out.println("Lagu dihapus : "+head_2019.judul_2019);

        //jika hanya satu node
        if(head_2019==tail_2019) {
            head_2019=null;
            tail_2019=null;
        }
        else {
            head_2019=head_2019.next_2019;
            head_2019.prev_2019=null;
        }
    }

    //fungsi tampil maju
    public static void tampilMaju_2019() {

        //jika kosong
        if(head_2019==null) {
            System.out.println("Playlist kosong");
            return;
        }

        Lagu_2511532019 curr_2019=head_2019;

        System.out.println("Playlist Maju:");

        while(curr_2019!=null) {
            System.out.println("Judul      :
"+curr_2019.judul_2019 + "("+ curr_2019.penyanyi_2019+ ")");

            curr_2019=curr_2019.next_2019;
        }
    }

    //fungsi tampil mundur
    public static void tampilMundur_2019() {

        //jika kosong
        if(tail_2019==null) {
            System.out.println("Playlist kosong");
            return;
        }

        Lagu_2511532019 curr_2019=tail_2019;

        System.out.println("Playlist Mundur:");
    }

```



```

        while(curr_2019!=null) {
            System.out.println("Judul      :
"+curr_2019.judul_2019 + "(" + curr_2019.penyanyi_2019 + ")");

            curr_2019=curr_2019.prev_2019;
        }
    }

    //fungsi cari lagu
    public static void cariLagu_2019(String judulCari_2019) {

        //jika kosong
        if(head_2019==null) {
            System.out.println("Playlist kosong");
            return;
        }

        Lagu_2511532019 curr_2019=head_2019;
        boolean ketemu_2019=false;

        while(curr_2019!=null) {

            if(curr_2019.judul_2019.equalsIgnoreCase(judulCari_2019)) {
                System.out.println("Lagu ditemukan");
                System.out.println("Judul      :
"+curr_2019.judul_2019);
                System.out.println("Penyanyi :
"+curr_2019.penyanyi_2019);

                ketemu_2019=true;
                break;
            }

            curr_2019=curr_2019.next_2019;
        }

        if(ketemu_2019==false) {
            System.out.println("Lagu tidak ditemukan");
        }
    }

    public static void main(String[] args) {

        Scanner input_2019=new Scanner(System.in);

        int pilih_2019;

        do {
            System.out.println("\n=== Playlist Musik NIM:
2511532019 ===");
            System.out.println("1. Tambah Lagu");
            System.out.println("2. Hapus Lagu Pertama");
            System.out.println("3. Lihat Playlist (Maju)");
            System.out.println("4. Lihat Playlist (Mundur)");
            System.out.println("5. Cari Lagu");

```

```

        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilih_2019=input_2019.nextInt();
        input_2019.nextLine();

        switch(pilih_2019) {

        case 1:
            System.out.print("Judul: ");
            String judul_2019=input_2019.nextLine();

            System.out.print("Penyanyi: ");
            String penyanyi_2019=input_2019.nextLine();

            tambahLagu_2019(judul_2019, penyanyi_2019);
            break;

        case 2:
            hapusLaguPertama_2019();
            break;

        case 3:
            tampilMaju_2019();
            break;

        case 4:
            tampilMundur_2019();
            break;

        case 5:
            System.out.print("Masukkan judul lagu: ");
            String cari_2019=input_2019.nextLine();

            cariLagu_2019(cari_2019);
            break;

        case 6:
            System.out.println("Program selesai");
            break;

        default:
            System.out.println("Pilihan tidak valid");
        }

    }while(pilih_2019!=6);

    input_2019.close();
}
}

```

```

public class PlaylistMusik_2511532019 {

    static Lagu_2511532019 head_2019=null;
    static Lagu_2511532019 tail_2019=null;

```

- `public class PlaylistMusik_2511532019 {`
→ Membuat class bernama PlaylistMusik_2511532019.
- `static Lagu_2511532019 head_2019=null;`
→ Pointer head untuk menunjuk node pertama DLL.
- `static Lagu_2511532019 tail_2019=null;`
→ Pointer tail untuk menunjuk node terakhir DLL.

```
//fungsi tambah lagu
public static void tambahLagu_2019(String judul_2019, String
penyanyi_2019) {

    Lagu_2511532019 newLagu_2019=
        new Lagu_2511532019(judul_2019,
penyanyi_2019);

    //jika DLL kosong
    if(head_2019==null) {
        head_2019=newLagu_2019;
        tail_2019=newLagu_2019;
    }
    else {
        tail_2019.next_2019=newLagu_2019;
        newLagu_2019.prev_2019=tail_2019;
        tail_2019=newLagu_2019;
    }

    System.out.println("Lagu berhasil ditambahkan!");
}
```

- `public static void tambahLagu_2019(...) {`
→ Method untuk menambahkan lagu ke playlist.
- `Lagu_2511532019 newLagu_2019=...`
→ Membuat node lagu baru.
- `new Lagu_2511532019(...)`
→ Memanggil konstruktor class Lagu_2511532019.
- `if(head_2019==null) {`
→ Mengecek apakah playlist kosong.
- `head_2019=newLagu_2019;`
→ Node baru menjadi head.
- `tail_2019=newLagu_2019;`
→ Node baru menjadi tail.}

- else {
→Kondisi jika DLL tidak kosong.
- tail_2019.next_2019=newLagu_2019;
→Tail lama dihubungkan ke node baru.
- newLagu_2019.prev_2019=tail_2019;
→Pointer prev node baru diarahkan ke tail lama.
- tail_2019=newLagu_2019;}
→Tail dipindahkan ke node baru.
- System.out.println("Lagu berhasil ditambahkan!");
→Menampilkan pesan lagu berhasil ditambahkan.}

```
//fungsi hapus lagu pertama
public static void hapusLaguPertama_2019() {

    //jika kosong
    if(head_2019==null) {
        System.out.println("Playlist kosong");
        return;
    }

    System.out.println("Lagu dihapus : "+head_2019.judul_2019);

    //jika hanya satu node
    if(head_2019==tail_2019) {
        head_2019=null;
        tail_2019=null;
    }
    else {
        head_2019=head_2019.next_2019;
        head_2019.prev_2019=null;
    }
}
```

- public static void hapusLaguPertama_2019() {
→Method menghapus node pertama.
- if(head_2019==null) {
→Mengecek apakah playlist kosong.
- System.out.println("Playlist kosong");
→Menampilkan pesan playlist kosong.
- return;}
→Menghentikan method.

- `System.out.println(...)`
→ Menampilkan judul lagu yang dihapus.
- `if(head_2019==tail_2019) {`
→ Mengecek apakah hanya ada satu node.
- `head_2019=null;`
→ Head dikosongkan.
- `tail_2019=null;}`
→ Tail dikosongkan.
- `else {`
→ Kondisi jika node lebih dari satu.
- `head_2019=head_2019.next_2019;`
→ Head dipindahkan ke node berikutnya.
- `head_2019.prev_2019=null;}}`
→ Pointer prev head baru dibuat null.

```
//fungsi tampil maju
public static void tampilMaju_2019() {

    //jika kosong
    if(head_2019==null) {
        System.out.println("Playlist kosong");
        return;
    }

    Lagu_2511532019 curr_2019=head_2019;

    System.out.println("Playlist Maju:");

    while(curr_2019!=null) {
        System.out.println("Judul      :
"+curr_2019.judul_2019 + "("+ curr_2019.penyanyi_2019+ ")");

        curr_2019=curr_2019.next_2019;
    }
}
```

- `public static void tampilMaju_2019() {`
→ Method menampilkan playlist dari depan.
- `if(head_2019==null) {`
→ Mengecek apakah playlist kosong.
- `System.out.println("Playlist kosong");`
→ Menampilkan pesan playlist kosong.

- return;}
→Menghentikan method.
- Lagu_2511532019 curr_2019=head_2019;
→Pointer traversal dimulai dari head.
- System.out.println("Playlist Maju:");
→Menampilkan judul traversal maju.
- while(curr_2019!=null) {
→Perulangan traversal selama node tidak null.
- System.out.println(...)
→Menampilkan judul dan penyanyi lagu.
- curr_2019=curr_2019.next_2019;}}
- Pointer berpindah ke node berikutnya.

```
//fungsi tampil mundur
public static void tampilMundur_2019() {

    //jika kosong
    if(tail_2019==null) {
        System.out.println("Playlist kosong");
        return;
    }

    Lagu_2511532019 curr_2019=tail_2019;

    System.out.println("Playlist Mundur:");

    while(curr_2019!=null) {
        System.out.println("Judul      :
"+curr_2019.judul_2019 + "(" + curr_2019.penyanyi_2019 + ")");

        curr_2019=curr_2019.prev_2019;
    }
}
```

- public static void tampilMundur_2019() {
→Method menampilkan playlist dari belakang.
- if(tail_2019==null) {
→Mengecek apakah playlist kosong.
- System.out.println("Playlist kosong");
→Menampilkan pesan playlist kosong.
- return;}
→Menghentikan method.

- Lagu_2511532019 curr_2019=tail_2019;
→Pointer traversal dimulai dari tail.
- System.out.println("Playlist Mundur:");
→Menampilkan judul traversal mundur.
- while(curr_2019!=null) {
→Perulangan traversal mundur.
- System.out.println(...)
→Menampilkan judul dan penyanyi lagu.
- curr_2019=curr_2019.prev_2019;}}
- Pointer berpindah ke node sebelumnya.

```
//fungsi cari lagu
public static void cariLagu_2019(String judulCari_2019) {

    //jika kosong
    if(head_2019==null) {
        System.out.println("Playlist kosong");
        return;
    }

    Lagu_2511532019 curr_2019=head_2019;
    boolean ketemu_2019=false;

    while(curr_2019!=null) {

        if(curr_2019.judul_2019.equalsIgnoreCase(judulCari_2019)) {
            System.out.println("Lagu ditemukan");
            System.out.println("Judul      :
"+curr_2019.judul_2019);
            System.out.println("Penyanyi :
"+curr_2019.penyanyi_2019);

            ketemu_2019=true;
            break;
        }

        curr_2019=curr_2019.next_2019;
    }

    if(ketemu_2019==false) {
        System.out.println("Lagu tidak ditemukan");
    }
}
```

- public static void cariLagu_2019(...) {
→Method mencari lagu berdasarkan judul.

- `if(head_2019==null) {`
→Mengecek apakah playlist kosong.
- `System.out.println("Playlist kosong");`
→Menampilkan pesan playlist kosong.
- `return;}`
→Menghentikan method.
- `Lagu_2511532019 curr_2019=head_2019;`
→Pointer traversal dimulai dari head.
- `boolean ketemu_2019=false;`
→Variabel penanda apakah lagu ditemukan.
- `while(curr_2019!=null) {`
→Perulangan traversal playlist.
- `if(curr_2019.judul_2019.equalsIgnoreCase(...)) {`
→Mengecek apakah judul lagu sama tanpa membedakan huruf besar kecil.
- `System.out.println("Lagu ditemukan");`
→Menampilkan pesan lagu ditemukan.
- `System.out.println("Judul : ...");`
→Menampilkan judul lagu.
- `System.out.println("Penyanyi : ...");`
→Menampilkan nama penyanyi.
- `ketemu_2019=true;`
→Mengubah status menjadi ditemukan.
- `break;}`
→Menghentikan perulangan.
- `curr_2019=curr_2019.next_2019;}`
→Pointer berpindah ke node berikutnya.
- `if(ketemu_2019==false) {`
→Mengecek apakah lagu tidak ditemukan.
- `System.out.println("Lagu tidak ditemukan");}}`
→Menampilkan pesan lagu tidak ditemukan.

```
public static void main(String[] args) {
```



```

Scanner input_2019=new Scanner(System.in);

int pilih_2019;

do {
    System.out.println("\n=== Playlist Musik NIM:
2511532019 ===");
    System.out.println("1. Tambah Lagu");
    System.out.println("2. Hapus Lagu Pertama");
    System.out.println("3. Lihat Playlist (Maju)");
    System.out.println("4. Lihat Playlist (Mundur)");
    System.out.println("5. Cari Lagu");
    System.out.println("6. Keluar");
    System.out.print("Pilihan: ");
    pilih_2019=input_2019.nextInt();
    input_2019.nextLine();
}

```

- public static void main(String[] args) {
→Method utama program.
- Scanner input_2019=new Scanner(System.in);
→Membuat objek input keyboard.
- int pilih_2019;
→Variabel pilihan menu.
- do {
→Awal perulangan menu.
- System.out.println(...)
→Menampilkan judul menu program.
- System.out.println("1. Tambah Lagu");
→Menampilkan menu tambah lagu.
- System.out.println("2. Hapus Lagu Pertama");
→Menampilkan menu hapus lagu pertama.
- System.out.println("3. Lihat Playlist (Maju)");
→Menampilkan menu traversal maju.
- System.out.println("4. Lihat Playlist (Mundur)");
→Menampilkan menu traversal mundur.
- System.out.println("5. Cari Lagu");
→Menampilkan menu pencarian lagu.
- System.out.println("6. Keluar");
→Menampilkan menu keluar.

- `System.out.print("Pilihan: ");`
→ Meminta input pilihan menu.
- `pilih_2019=input_2019.nextInt();`
→ Membaca input angka pilihan.
- `input_2019.nextLine();`
→ Membersihkan enter yang tersisa di buffer.

```

switch(pilih_2019) {
    case 1:
        System.out.print("Judul: ");
        String judul_2019=input_2019.nextLine();

        System.out.print("Penyanyi: ");
        String penyanyi_2019=input_2019.nextLine();

        tambahLagu_2019(judul_2019, penyanyi_2019);
        break;

    case 2:
        hapusLaguPertama_2019();
        break;

    case 3:
        tampilMaju_2019();
        break;

    case 4:
        tampilMundur_2019();
        break;

    case 5:
        System.out.print("Masukkan judul lagu: ");
        String cari_2019=input_2019.nextLine();

        cariLagu_2019(cari_2019);
        break;

    case 6:
        System.out.println("Program selesai");
        break;

    default:
        System.out.println("Pilihan tidak valid");
}

}while(pilih_2019!=6);

input_2019.close();
}

```

- `switch(pilih_2019) {`
→Memilih proses berdasarkan menu yang dipilih user.
- `case 1:`
→Pilihan menu untuk menambah lagu.
- `System.out.print("Judul: ");`
→Meminta input judul lagu.
- `String judul_2019=input_2019.nextLine();`
→Menyimpan input judul lagu ke variabel `judul_2019`.
- `System.out.print("Penyanyi: ");`
→Meminta input nama penyanyi.
- `String penyanyi_2019=input_2019.nextLine();`
→Menyimpan input penyanyi ke variabel `penyanyi_2019`.
- `tambahLagu_2019(judul_2019, penyanyi_2019);`
→Memanggil method untuk menambahkan lagu ke playlist.
- `break;`
→Menghentikan case agar tidak masuk ke case berikutnya.
- `case 2:`
→Pilihan menu untuk menghapus lagu pertama.
- `hapusLaguPertama_2019();`
→Memanggil method menghapus node pertama.
- `break;`
→Menghentikan case.
- `case 3:`
→Pilihan menu menampilkan playlist maju.
- `tampilMaju_2019();`
→Memanggil method traversal maju.
- `break;`
→Menghentikan case.
- `case 4:`
→Pilihan menu menampilkan playlist mundur.
- `tampilMundur_2019();`
→Memanggil method traversal mundur.

- `break;`
→Menghentikan case.
- case 5:
→Pilihan menu mencari lagu.
- `System.out.print("Masukkan judul lagu: ");`
→Meminta input judul lagu yang dicari.
- `String cari_2019=input_2019.nextLine();`
→Menyimpan input judul lagu ke variabel `cari_2019`.
- `cariLagu_2019(cari_2019);`
→Memanggil method pencarian lagu.
- `break;`
→Menghentikan case.
- case 6:
→Pilihan menu keluar program.
- `System.out.println("Program selesai");`
→Menampilkan pesan program selesai.
- `break;`
→Menghentikan case.
- default:
→Dijalankan jika input menu tidak tersedia.
- `System.out.println("Pilihan tidak valid");}`
→Menampilkan pesan input salah.
- `}while(pilih_2019!=6);`
→Perulangan akan terus berjalan selama pilihan bukan 6.
- `input_2019.close();}`
→Menutup objek scanner.

Output

```
=== Playlist Musik NIM: 2511532019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: monokrom
Penyanyi: tulus
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511532019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: count on me
Penyanyi: bruno mars
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511532019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: a million dream
Penyanyi: pink
Lagu berhasil ditambahkan!
```

```
=== Playlist Musik NIM: 2511532019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: laskar pelangi
Penyanyi: nidji
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511532019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
Playlist Maju:
Judul    : monokrom(tulus)
Judul    : count on me(bruno mars)
Judul    : a million dream(pink)
Judul    : laskar pelangi(nidji)
```

```

=== Playlist Musik NIM: 2511532019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
Playlist Mundur:
Judul   : laskar pelangi(nidji)
Judul   : a million dream(pink)
Judul   : count on me(bruno mars)
Judul   : monokrom(tulus)

=== Playlist Musik NIM: 2511532019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: a million dream
Lagu ditemukan
Judul   : a million dream
Penyanyi : pink

=== Playlist Musik NIM: 2511532019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu dihapus : monokrom

```

```

=== Playlist Musik NIM: 2511532019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu dihapus : count on me

=== Playlist Musik NIM: 2511532019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: count on me
Lagu tidak ditemukan

=== Playlist Musik NIM: 2511532019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
Playlist Maju:
Judul   : a million dream(pink)
Judul   : laskar pelangi(nidji)

```

```

=== Playlist Musik NIM: 2511532019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Program selesai

```